Array creation

See also:
Array creation routines (../reference/routines.array-creation.html#routines-array-creation)

Introduction

There are 5 general mechanisms for creating arrays:

1. Conversion from other Python structures (e.g., lists, tuples)
2. Intrinsic numpy array array creation objects (e.g., arange, ones, zeros, etc.)
3. Reading arrays from disk, either from standard or custom formats
4. Creating arrays from raw bytes through the use of strings or buffers
5. Use of special library functions (e.g., random)

This section will not cover means of replicating, joining, or otherwise expanding or mutating existing arrays. Nor will it cover creating object arrays or record arrays. Both of those are covered in their own sections.

Converting Python array_like Objects to Numpy Arrays

In general, numerical data arranged in an array-like structure in Python can be converted to arrays through the use of the array() function. The most obvious examples are lists and tuples. See the documentation for array() for details for its use. Some objects may support the array-protocol and allow conversion to arrays this way. A simple way to find out if the object can be converted to a numpy array using array() is simply to try it interactively and see if it works! (The Python Way).

Examples:

```
>>>
>>> x = np.array([2,3,1,0])
>>> x = np.array([2, 3, 1, 0])
>>> x = np.array([[1,2.0],[0,0],(1+1j,3.)]) # note mix of tuple and lists,
    and types
>>> x = np.array([[ 1.+0.j, 2.+0.j], [ 0.+0.j, 0.+0.j], [ 1.+1.j, 3.+0.j]])
```

Intrinsic Numpy Array Creation

Numpy has built-in functions for creating arrays from scratch:

zeros(shape) will create an array filled with 0 values with the specified shape. The default dtype is float64.

```
>>> np.zeros((2, 3)) array([[ 0., 0., 0.], [ 0., 0., 0.]])
```

ones(shape) will create an array filled with 1 values. It is identical to zeros in all other respects.

arange() will create arrays with regularly incrementing values. Check the docstring for complete information on the various ways it can be used. A few examples will be given here:

```
>>> np.arange(10)
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.arange(2, 10, dtype=np.float)
array([ 2., 3., 4., 5., 6., 7., 8., 9.])
>>> np.arange(2, 3, 0.1)
array([ 2. , 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9])
```

Note that there are some subtleties regarding the last usage that the user should be aware of that are described in the arange docstring.

linspace() will create arrays with a specified number of elements, and spaced equally between the specified beginning and end values. For example:

```
>>> np.linspace(1., 4., 6)
array([ 1. ,  1.6,  2.2,  2.8,  3.4,  4. ])
```

The advantage of this creation function is that one can guarantee the number of elements and the starting and end point, which arange() generally will not do for arbitrary start, stop, and step values.

indices() will create a set of arrays (stacked as a one-higher dimensioned array), one per dimension with each representing variation in that dimension. An example illustrates much better than a verbal description:

```
>>> np.indices((3,3))
array([[[0, 0, 0], [1, 1, 1], [2, 2, 2]], [[0, 1, 2], [0, 1, 2], [0, 1, 2]]])
```

This is particularly useful for evaluating functions of multiple dimensions on a regular grid.

Reading Arrays From Disk

This is presumably the most common case of large array creation. The details, of course, depend greatly on the format of data on disk and so this section can only give general pointers on how to handle various formats.

Standard Binary Formats

Various fields have standard formats for array data. The following lists the ones with known python libraries to read them and return numpy arrays (there may be others for which it is possible to read and convert to numpy arrays so check the last section as well)

```
HDF5: PyTables
FITS: PyFITS
```

Examples of formats that cannot be read directly but for which it is not hard to convert are those formats supported by libraries like PIL (able to read and write many image formats such as jpg, png, etc).

Common ASCII Formats

Comma Separated Value files (CSV) are widely used (and an export and import option for programs like Excel). There are a number of ways of reading these files in Python. There are CSV functions in Python and functions in pylab (part of matplotlib).

More generic ascii files can be read using the io package in scipy.

Custom Binary Formats

There are a variety of approaches one can use. If the file has a relatively simple format then one can write a simple I/O library and use the numpy fromfile() function and .tofile() method to read and write numpy arrays directly (mind your byteorder though!) If a good C or C++ library exists that read the data, one can wrap that library with a variety of techniques though that certainly is much more work and requires significantly more advanced knowledge to interface with C or C++.

Use of Special Libraries

There are libraries that can be used to generate arrays for special purposes and it isn't possible to enumerate all of them. The most common uses are use of the many array generation functions in random that can generate arrays of random values, and some utility functions to generate special matrices (e.g. diagonal).

Previous topic

Data types (basics.types.html)

Next topic

I/O with Numpy (basics.io.html)